

# Applied AI Assignment 1

---

**Submitted By:**

Eman Khalid 22i-XXXX

Moimma Ali Khan 22i-XXXX

**Date**

17/02/2025.

Spring 2024



**Department of Software Engineering**

FAST – National University of Computer & Emerging Sciences

|                                         |           |
|-----------------------------------------|-----------|
| <b>Approach</b>                         | <b>3</b>  |
| <b>Algorithms</b>                       | <b>9</b>  |
| Breadth-First Search (BFS)              | 9         |
| Purpose                                 | 9         |
| How it works                            | 9         |
| Uniform Cost Search (UCS)               | 10        |
| Purpose                                 | 10        |
| How it works                            | 10        |
| A* Algorithm                            | 10        |
| Purpose                                 | 10        |
| How it works                            | 11        |
| Comparison of all Algorithms            | 11        |
| <b>Challenges during Implementation</b> | <b>11</b> |
| <b>Appendix</b>                         | <b>13</b> |

# Approach

We broke our implementation down into a series of steps that we implemented turn by turn. Here are the stages of our implementation.

- First we needed a dataset for words, we used a dictionary.txt file containing exactly 502162 words to choose from and which would be used for different game modes (beginner, advanced and challenge) later on.

```
#load words frm dictonay
def load_words(filename="dictionary.txt"):
    """Load words from a predefined dictionary file."""
    try:
        with open(filename, "r", encoding="utf-8") as file:
            words = {line.strip().lower() for line in file if line.strip()}
            #print(f"Loaded words: {words}") # Debug print
            print(f"Loaded {len(words)} words from {filename}.") # More concise

            return words
    except FileNotFoundError:
        print(f"Error: {filename} not found.")
        return set()
```

- We created a build graph function as one of the main pre-requisites for the assignment was to implement graph-based visualisation for our results/paths. For this we defined the following function.

```
def build_graph(word_list):
    """Build a word transformation graph efficiently using adjacency buckets."""
    graph = defaultdict(list)
    adjacency_map = defaultdict(list)

    # Create pattern-based adjacency (e.g., "c_t" -> ["cat", "cut"])
    for word in word_list:
        for i in range(len(word)):
            pattern = word[:i] + "_" + word[i+1:]
            adjacency_map[pattern].append(word)

    # Populate the graph
    for words in adjacency_map.values():
        for word in words:
            graph[word].extend(w for w in words if w != word)
```

```

def visualize_graph(graph, path=None, filename="word_graph.png", degree_threshold=3):
    """Visualizes the word transformation graph with filtering for clarity."""
    G = nx.Graph()

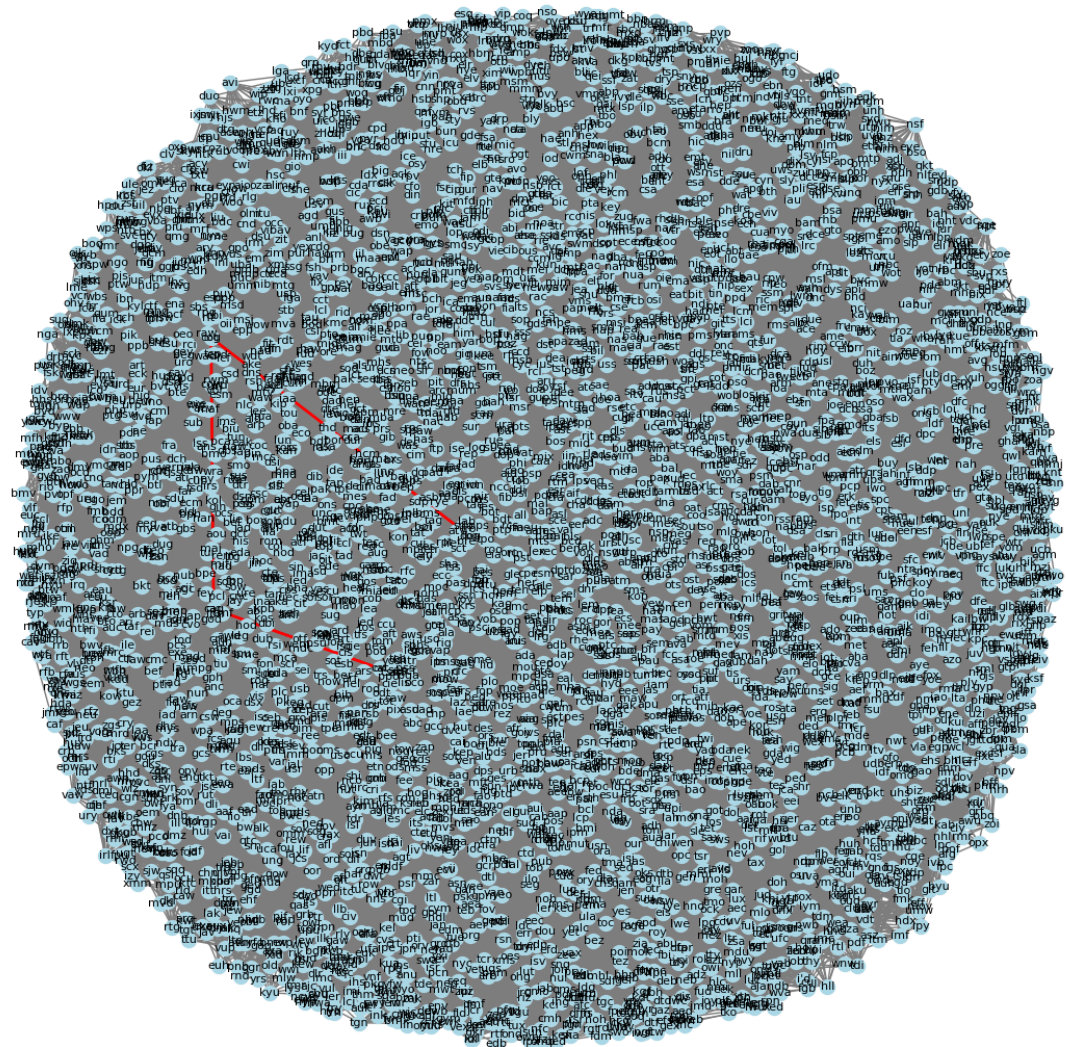
    # Add edges
    for word, neighbors in graph.items():
        for neighbor in neighbors:
            G.add_edge(word, neighbor)

    # Filter nodes with degree above threshold
    high_degree_nodes = [node for node, deg in G.degree() if deg > degree_threshold]
    G_sub = G.subgraph(high_degree_nodes)

    # Adjust layout for better spacing
    plt.figure(figsize=(12, 12))
    pos = nx.spring_layout(G_sub, k=0.6) # Increase k to spread nodes apart

    # Reduce label clutter (show only important nodes)
    labels = {node: node if node in path else "" for node in G_sub.nodes}

```



- Next we had to ensure that every other word in the graph differed by one letter exactly. To do this we defined a function right above the graph function like this.

```
# Check if two words differ by exactly one letter
def is_adjacent(word1, word2):
    return sum(1 for a, b in zip(word1, word2) if a != b) == 1
```

- The main purpose of this game is to serve as an educational tool to learn the three main searching algorithms specified, i.e. A\* algorithm, UCS, and BFS.

```

# Breadth-First Search (BFS)
def bfs(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    queue = deque([(start, [start])])
    visited = set()
    while queue:
        word, path = queue.popleft()
        if word == end:
            return path
        visited.add(word)
        for neighbor in graph.get(word, []): # Use .get() to prevent KeyError
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))
    return None

```

```

# Uniform Cost Search (UCS)
def ucs(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, word, path = heapq.heappop(pq)
        if word == end:
            return path
        if word not in visited:
            visited.add(word)
            for neighbor in graph.get(word, []):
                heapq.heappush(pq, (cost + 1, neighbor, path + [neighbor]))
    return None

```

```
# A* Search
def heuristic(word, end):
    return sum(1 for a, b in zip(word, end) if a != b)

def a_star(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    pq = [(heuristic(start, end), 0, start, [start])]
    visited = set()
    while pq:
        _, cost, word, path = heapq.heappop(pq)
        if word == end:
            return path
        if word not in visited:
            visited.add(word)
            for neighbor in graph.get(word, []):
                heapq.heappush(pq, (cost + heuristic(neighbor, end), cost + 1, neighbor, path + [neighbor]))
    return None
```

- Our gameplay features two main play modes, each with three difficulty levels. The first is manual gameplay, where the player decides to climb the ladder using an algorithm of choice, and the second is AI-assisted gameplay, where the player has an AI hint system to help them with every guess. This hint system is uniquely implemented by giving the user a definition of the word they should guess. We did this by calling a free dictionary api as shown.

```
# Function to fetch the definition of a word from Free Dictionary API
def get_word_definition(word):
    url = f"https://api.dictionaryapi.dev/api/v2/entries/en/{word}"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            data = response.json()
            if isinstance(data, list) and "meanings" in data[0]:
                meanings = data[0]["meanings"]
                if meanings:
                    return meanings[0]["definitions"][0]["definition"]
            return "No definition found."
    except Exception as e:
        return f"Error fetching definition: {e}"
```

- Manual Gameplay

```

Select Play Mode:
1. Manual Gameplay
2. AI-Assisted Gameplay
Enter mode (1/2): 1

Choose Search Algorithm:
1. BFS
2. UCS
3. A* Search
Enter choice (1/2/3): 2

♦ You must transform 'cat' → 'dog'. Guess each word in the path!
You have 3 incorrect attempts for the entire game.
Enter your name: cag
♦ What is the next word after 'cat'? (3 attempts left): cag
✅ Correct! Moving to the next word...
♦ What is the next word after 'cag'? (3 attempts left): cog
✅ Correct! Moving to the next word...
♦ What is the next word after 'cog'? (3 attempts left): dog
✅ Correct! Moving to the next word...

🎉 Congratulations! You reached the final word successfully!
🔗 Path followed: cat -> cag -> cog -> dog
🏆 Final Score: 30 points
🏆 cag, you earned 30 points.

```

- AI Hinting System

```

♦ AI Hint System Active! Transform 'cat' → 'dog'. You have 3 total incorrect attempts.
Enter your name: emkay

Hint 1: The next word has this meaning → No definition found.
♦ Guess the word (Incorrect attempts left: 3): cag
✅ Correct! Moving to the next word...

Hint 2: The next word has this meaning → A tooth on a gear.
♦ Guess the word (Incorrect attempts left: 3): ch
❌ Incorrect! (-5 points) (2 attempts left)
♦ Guess the word (Incorrect attempts left: 2): cog
✅ Correct! Moving to the next word...

Hint 3: The next word has this meaning → A mammal, Canis familiaris or Canis lupus familiaris, that has been do
♦ Guess the word (Incorrect attempts left: 2): dog
✅ Correct! Moving to the next word...

🎉 Congratulations! You reached the final word successfully!
🔗 Path followed: cat → cag → cog → dog
🏆 Final Score: 25 points
🏆 emkay, you earned 25 points.

```



# Algorithms

We used 3 main algorithms as follows:

## Breadth-First Search (BFS)

### Purpose

BFS is used to find the shortest path (in terms of the number of transformations) from the start word to the target word.

### How it works

- Uses a **queue (FIFO)** to explore words level by level.
- Each word's neighbors (valid one-letter transformations) are added to the queue.
- The first time the target word is reached, BFS guarantees the shortest path.

```
# Breadth-First Search (BFS)
def bfs(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    queue = deque([(start, [start])])
    visited = set()
    while queue:
        word, path = queue.popleft()
        if word == end:
            return path
        visited.add(word)
        for neighbor in graph.get(word, []): # Use .get() to prevent KeyError
            if neighbor not in visited:
                queue.append((neighbor, path + [neighbor]))
    return None
```

# Uniform Cost Search (UCS)

## Purpose

UCS finds the **least-cost path** from the start word to the target word. In this case, all moves have equal cost (1 per transformation), so UCS behaves like BFS.

## How it works

- Uses a **priority queue (min-heap)**
- Always expands the **cheapest** path first
- Ensures an **optimal solution** in terms of the number of moves.

```
# Uniform Cost Search (UCS)
def ucs(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    pq = [(0, start, [start])]
    visited = set()
    while pq:
        cost, word, path = heapq.heappop(pq)
        if word == end:
            return path
        if word not in visited:
            visited.add(word)
            for neighbor in graph.get(word, []):
                heapq.heappush(pq, (cost + 1, neighbor, path + [neighbor]))
    return None
```

# A\* Algorithm

## Purpose

A\* finds the **shortest path efficiently** by using a heuristic function to prioritize promising paths.

## How it works

- Uses a **priority queue** (like UCS).
- Prioritizes paths based on **cost so far + estimated cost to goal**.
- The heuristic is the number of different letters between the current word and the target word.

```
# A* Search
def heuristic(word, end):
    return sum(1 for a, b in zip(word, end) if a != b)

def a_star(start, end, graph):
    if start not in graph or end not in graph:
        print(f"Error: '{start}' or '{end}' not in graph.")
        return None

    pq = [(heuristic(start, end), 0, start, [start])]
    visited = set()
    while pq:
        _, cost, word, path = heapq.heappop(pq)
        if word == end:
            return path
        if word not in visited:
            visited.add(word)
            for neighbor in graph.get(word, []):
                heapq.heappush(pq, (cost + heuristic(neighbor, end), cost + 1, neighbor, path + [neighbor]))
    return None
```

## Comparison of all Algorithms

- BFS: Works well for short word ladders since all transformations cost equally.
- UCS: Not much benefit over BFS since all word transitions have the same cost.
- A\*: Best choice, as the heuristic helps prioritize promising paths, making it faster for longer transformations.

## Challenges during Implementation

The challenges we faced were as follows:

- The word ladder problem requires a graph where each node (word) is connected to all valid one-letter transformations. Building this graph efficiently is tricky, especially for large dictionaries (e.g., 500,000+ words in dictionary.txt). To solve this problem, we ended up using an adjacency list, where each word is mapped to all possible words that differ by one letter.

- A\* requires a good heuristic to estimate the distance from the current word to the target. A poorly chosen heuristic could make A\* behave like UCS (losing efficiency). Thus, we defined a simple heuristic function to give a rough estimate of how many transformations are still needed.

```
# A* Search
def heuristic(word, end):
    return sum(1 for a, b in zip(word, end) if a != b)
```

- The AI-generated hints sometimes failed to find dictionary definitions for intermediate words. We received hints like **"No definition found."**, which wasn't helpful. Some solutions which could be implemented in future could be to either integrate an offline dictionary, or use an external API like WordNet for better definitions.

# Appendix

- Source Code (ours)  
[https://colab.research.google.com/drive/1HAdaXBHfjWNYG\\_B628afO8FpPjxUBPOA#scrollTo=L0CqwWsRybjc](https://colab.research.google.com/drive/1HAdaXBHfjWNYG_B628afO8FpPjxUBPOA#scrollTo=L0CqwWsRybjc)
- Word definitions API for hinting system  
<https://pipedream.com/apps/dictionary-api>